

RELATÓRIO TÉCNICO

Otimização de rotas para distribuição de medicamentos e insumos

Pós-graduação em Inteligência Artificial para Desenvolvedores

A solução combina algoritmo genético, restrições logísticas, visualizações, geração de instruções com modelo de linguagem e experimentação no Azure Machine Learning. Este documento apresenta as decisões de implementação, os resultados e os limites da abordagem.

Conteúdo

1. Contexto e objetivo
2. Modelagem do problema
3. Algoritmo genético
4. Tratamento das restrições
5. Comparação de abordagens
6. Visualização e análise
7. Integração com modelo de linguagem
8. Arquitetura e infraestrutura
9. Testes e reprodutibilidade
10. Limitações, validade e uso responsável
11. Conclusão

1. Contexto e objetivo

A distribuição entre unidades hospitalares e pontos de atendimento domiciliar exige que demandas com urgências distintas sejam atendidas com uma frota limitada. O objetivo da solução é reduzir o percurso total sem perder de vista a prioridade clínica, a capacidade de carga e a autonomia de cada veículo. O cenário utilizado é fictício e contém um depósito, dez pontos de entrega e três veículos.

1.1 Método de desenvolvimento e avaliação

O desenvolvimento partiu da formulação didática do TSP: representação por permutação, matriz de distâncias, função de aptidão, seleção, crossover ordenado, mutação e acompanhamento da convergência. A evolução para o contexto hospitalar ocorreu em incrementos verificáveis: prioridades; capacidade e autonomia; e, por fim, a decodificação da permutação em uma rota para cada veículo.

A análise combina testes automatizados, algoritmos de referência, repetições com sementes distintas e inspeção visual das rotas e da convergência. Os resultados numéricos se referem ao cenário fictício versionado em `data/deliveries.json`.

2. Modelagem do problema

O problema clássico do caixeiro viajante procura a menor rota que visita cada ponto uma única vez e retorna à origem. A formulação é ampliada para um problema de roteamento de veículos: cada entrega pertence a exatamente uma rota e toda rota começa e termina no depósito.

Cada entrega possui coordenadas, demanda em quilogramas, tempo de serviço e prioridade de 1 (regular) a 3 (crítica). Cada veículo possui capacidade e autonomia. As distâncias são calculadas pela fórmula de Haversine e armazenadas em uma matriz de adjacência. Trata-se de distância geodésica; trânsito, sentido das vias e tempo real de deslocamento não são representados.

3. Algoritmo genético

3.1 Representação e população inicial

Um indivíduo é uma permutação dos índices das entregas. Essa codificação impede repetição ou omissão de pontos. Um decodificador percorre o cromossomo e atribui cada entrega ao veículo de menor custo projetado, considerando as proporções de carga e autonomia e dando preferência a alocações sem violação.

A população combina indivíduos aleatórios com dois hotstarts: prioridades em ordem decrescente e vizinho mais próximo ajustado pela prioridade. A semente pseudoaleatória é configurável.

3.2 Aptidão

Como o problema é de minimização, valores menores são melhores: $f(x) = D + \alpha P + \beta Ec + \gamma Ea$. D é a distância total; P é a distância acumulada até cada parada, ponderada pela prioridade; Ec e Ea são excessos de capacidade e autonomia. Os coeficientes padrão são $\alpha=0,35$, $\beta=1.000$ e $\gamma=1.000$.

3.3 Operadores e término

A seleção é feita por torneio. O Ordered Crossover (OX1) preserva uma permutação válida. A mutação alterna entre troca de dois genes e inversão de segmento. O elitismo preserva os melhores indivíduos e pode ser configurado como zero. A execução termina ao atingir o limite de gerações ou o número configurado de gerações sem melhora.

O histórico registra melhor aptidão, média e desvio-padrão por geração. A visualização separa melhor fitness e fitness médio em painéis distintos.

4. Tratamento das restrições

- Prioridade: entregas críticas recebem custo maior conforme aumenta a distância percorrida antes do atendimento.

- Carga: a demanda total da rota é confrontada com a capacidade específica do veículo; excessos são penalizados e apresentados no resultado.
- Autonomia: o percurso completo, incluindo o retorno, é comparado com a distância máxima do veículo.
- Múltiplos veículos: o decodificador cria uma rota por veículo e aloca cada entrega uma única vez.

As restrições são flexíveis por penalização. Isso permite diagnosticar cenários sem solução, mas não prova viabilidade antes da busca. Janelas de tempo e carga fracionada não são tratadas nesta versão.

5. Comparação de abordagens

Abordagem	Vantagem	Limitação	Uso
Força bruta	Encontra o ótimo no espaço avaliado	Crescimento fatorial $O(n!)$	Até nove entregas
Vizinho mais próximo	Muito rápido e simples	Pode produzir rota ruim	Baseline e hotstart
Algoritmo genético	Explora espaço amplo e aceita restrições	Não garante ótimo global	Cenário completo

As comparações usam a mesma função de aptidão e a mesma instância. Tempo e economia monetária somente podem ser calculados quando houver dados históricos ou medições reais.

5.1 Protocolo experimental e resultados

No cenário completo, o algoritmo foi executado cinco vezes com população 80, limite de 150 gerações, estagnação de 50 gerações e sementes 7, 21, 42, 84 e 123. O vizinho mais próximo obteve fitness 150,17.

Semente	Fitness	Distância (km)	Gerações	Redução frente à baseline
7	123,97	67,58	75	17,55%
21	123,97	67,58	78	17,55%
42	126,92	65,34	79	15,58%
84	127,44	68,54	65	15,24%
123	123,97	67,58	101	17,55%

A média foi 125,25, com desvio-padrão populacional 1,58 e redução média de fitness de 16,70%. A menor distância não coincide necessariamente com o menor fitness porque a função também valoriza o atendimento antecipado de entregas prioritárias.

Em uma subinstância com oito entregas, força bruta e algoritmo genético alcançaram fitness 91,22 e distância 54,72 km. O vizinho mais próximo obteve fitness 123,99 e distância 72,62 km.

6. Visualização e análise

O mapa HTML apresenta depósito, sequência numerada das paradas e uma cor para cada veículo. A prioridade é identificada por legenda própria. A convergência é exibida em dois gráficos separados: melhor fitness e média da população. Esses artefatos apoiam a inspeção das rotas e da estabilização da busca.

7. Integração com modelo de linguagem

A camada de relatórios recebe a solução estruturada. O prompt define papel, tarefa, idioma e regras contra alucinação, seguido pelo contexto em JSON. A temperatura baixa (0,2) favorece respostas estáveis. A integração usa um modelo Gemini pré-treinado; não há treinamento local.

- GeminiReportGenerator: gera instruções, relatórios e respostas por meio da API.
- LocalReportGenerator: cria documento determinístico e permite execução sem rede ou chave.

As tarefas abrangem instruções por veículo, resumo diário, relatório semanal, sugestões e perguntas em linguagem natural. A saída não altera rotas nem decisões de viabilidade. Respostas generativas exigem revisão humana; dados pessoais e clínicos não devem ser enviados.

RAG não foi aplicado porque não existe uma coleção documental externa a recuperar. O aterramento ocorre diretamente pelo JSON da solução. Uma futura base de protocolos deverá ser versionada e recuperada com referência de origem.

8. Arquitetura e infraestrutura



O domínio não depende da visualização nem da LLM. Essa separação facilita testes e permite substituir o provedor generativo sem alterar o otimizador. O pacote enviado ao Azure ML é filtrado por `.amlignore`, que exclui `.env`, histórico Git, infraestrutura, notebooks, testes, dados e resultados locais.

8.1 Execução no Azure Machine Learning

O Azure Machine Learning Workspace funciona como ambiente de experimentação. O cenário JSON é registrado como ativo de dados versionado. Um command job executa o núcleo Python em computação serverless, registra parâmetros e métricas com MLflow e persiste solução, mapa, convergência, relatório e histórico no output artifacts. O job usa uma máquina `Standard_DS2_v2`.

Computação serverless também consome cota do Azure Machine Learning. Um `Standard_DS2_v2` requer duas vCPUs disponíveis na região. Com cota zero, a infraestrutura permanente pode ser criada, mas jobs e Compute Instances não executam até que a cota seja liberada.

O sweep usa busca aleatória sobre tamanho da população, taxas de crossover e mutação e elitismo, minimizando fitness. No notebook, essa etapa é opcional e vem desativada por padrão. Quando necessário, utiliza o cluster CPU de baixa prioridade `cpu-cluster`, com escala de zero a quatro nós.



O Terraform cria Resource Group, Storage Account, Key Vault, Log Analytics, Application Insights e Workspace. Por padrão, não cria o cluster, permitindo concluir o provisionamento mesmo sem cota de vCPU. Após a liberação da cota, o cluster pode ser habilitado com `create_compute_cluster=true`. As saídas do Terraform mostram assinatura, região, Resource Group e workspace para a configuração do notebook.

A chave Gemini não é incluída no IaC, nos manifestos nem no pacote de código. Sem `GEMINI_API_KEY` no ambiente remoto, o job produz o relatório determinístico local e mantém os demais artefatos. O notebook conduz registro dos ativos, submissão, acompanhamento, download e visualização dos resultados.

9. Testes e reprodutibilidade

A suíte contém 15 testes. Eles verificam distância, validações, preservação da permutação pelo OX1, visita única, penalização de rota inviável, efeito da prioridade, elitismo zero, reprodutibilidade, comparação com força bruta, fundamentação do prompt, respostas completas ou truncadas do Gemini e visualizações.

O ambiente local é descrito por `pyproject.toml` e `requirements.txt`; o ambiente remoto é definido pelos manifestos do Azure ML. A semente controla a aleatoriedade. O notebook principal reproduz o experimento local, enquanto o job em nuvem registra parâmetros, métricas e artefatos no MLflow.

10. Limitações, validade e uso responsável

- A distância geodésica não representa malha viária, trânsito, bloqueios ou tempo de atendimento.

- Uma solução inviável é um diagnóstico do cenário, não autorização para exceder limites operacionais.
- A força bruta é restrita a instâncias reduzidas e o algoritmo genético não garante ótimo global.
- Os dados são fictícios e não incluem pacientes; uso institucional exigiria controle de acesso, retenção definida e minimização de dados.
- O JSON da solução é a fonte de verdade; instruções da LLM precisam de validação operacional.

O processamento no Azure Machine Learning não representa treinamento de modelo preditivo. O algoritmo genético é um método de otimização combinatória, e o MLflow é usado para rastrear experimentos e hiperparâmetros.

11. Conclusão

A combinação de representação combinatória e decodificação multi-veículo preserva operadores próprios do TSP e incorpora restrições logísticas. A aptidão torna explícito o compromisso entre percurso, urgência e viabilidade. A integração generativa ocorre após a otimização e recebe dados estruturados. Para uso real, ainda seriam necessários matriz viária, tempos históricos, janelas de atendimento e custos operacionais.